

Using Jacobi Orthogonal Polynomials in Artificial Neural Network

Phi Duong
Florida Institute of technology
Melbourne Florida United States
pduong2018@my.fit.edu

1. Introduction

Traditional Artificial Neural Networks (ANN) can contain limitations on how the network can learn. Orthogonal polynomials provide a solution to one of these problems. Standard ANN models use linear base net values and a, generally, a sigmoid function to bound the output between zero and one. This limits the possible solution that a single layer can solve. For example, a simple ANN will only need 1 layer of units to represent AND, OR, NAND, NOR, but cannot represent XOR functions.

Orthogonal polynomials provide two main benefits. First, the function uses non-linearity, which allows the unit values to be more flexible in its representation. The second is the use of a set of polynomials that are orthogonal to each other. This allows each layer to act independent of one another, and thus, “helps to avoid column degeneracy of regression matrix” [2]. In addition, each polynomial in the set of orthogonal polynomials span out a different dimension in which the model can learn through. With these advantages, we expect to see an increase in the network’s ability and speed to learn more complicated networks with less layers and units needed. In addition, it adds a new layer in which the network can represent the data which should allow these models to generalize the data better. The downside of using these orthogonal polynomials is computational expenses. This paper will try to compare the differences in learning capabilities and computational expenses between a simple ANN and one with orthogonal polynomials

This paper uses one of the orthogonal polynomials, specifically the Jacobi polynomial set. A Jacobi polynomial can be characterized by the orthogonality rule

$$\int_{-1}^1 (1-x)^{\alpha}(1+x)^{\beta} J_m^{(\alpha,\beta)}(x) J_n^{(\alpha,\beta)}(x) dx = 0, \quad m \neq n \quad (1)$$

Where $P(x)$ is the polynomial function with α and β as non-negative hyper parameters that control the shape of the function, and the subscript n and m are the degree of the polynomial.

To better understand the ideas of orthogonality and its use, we will go into a bit of depth of how the function works and general equations that will be used in the program

2. Related work

In other works, Chakraborty [1], describes the theory and design uses for the Jacobi polynomials. Explains the use case of “define the Jacobi polynomials with multiple reference vectors for codes, and give the MacWilliams type identity for it”. Chu Kwong Chak and Chi Ming Chen’s paper on “Orthogonal polynomials neural networks for function approximation” [2] Shows a possible

re structure of the neural networks that work with orthogonal polynomials. H. Zhang and R. Yu’s paper on “Risk Assessment of substation based on Hermite orthogonal basis Feedforward Neural Network data fusion algorithm” [3] uses different orthogonal polynomials to research into, specifically the Legendre and Chebyshev polynomials. Their research uses MapReduce Parallelization to process large amounts of data. Though out of scope for this project, it shows some future improvements to these functions. Their main idea of using Orthogonal polynomials is to reduce “complex training iteration process, save a lot of time and converge fast.” This paper, using parallelization, shows huge improvements in computational time compared to the Backpropagation. Therefore, major improvements can also be applied to this paper’s network. Both Gabor Szego “Orthogonal polynomials” [6] and Shen, Jie, “Orthogonal polynomials and related approximation results” [7], shows the proofs and equations used in this paper approach. These algorithms allows possible work arounds to the general equation when applying the Jacobi Polynomials inside of a ANN.

3. Approach

We will define here the Jacobi polynomial set, and how it will be applied to the standard ANN. Functions and proofs will be generally explained for clarity but not rigorously proved out.

Orthogonality gives a more rigorous approach in defining the type of polynomials we want to use to map out the data. Orthogonality also increases search space by not allowing each polynomial to search through the same vector space, which should allow the model to search through the hypothesis space more efficiently.

3.1 Defining the Jacobi Polynomial Set

Let’s explain, generally, how orthogonal polynomials work. Orthogonal vectors are vectors in which their angles of direction are perpendicular to each other. We can define a vector $A = \langle a_0, a_1, \dots, a_n \rangle$ and another vector $B = \langle b_0, b_1, \dots, b_n \rangle$ are orthogonal if the dot product between vector is equal to zero. Similarly, we can define a polynomial as a set of vector values that is associated for each x values in the function.

The general Orthogonal polynomial equation is represented as

$$\int_a^b J_n(x) J_m(x) w(x) dx = 0, \quad m \neq n \quad (2)$$

Where $J_n(x)$ is a polynomial function with a degree of n . The integral acts at the vector sum for the product of the two different polynomials. $W(x)$ is considered as the weight of the function as

long as $\int_a^b w(x) dx > 0$. Gabor Szego [6] on “Orthogonal Polynomials,” explains the proof on how this shows orthogonality. Changing $w(x)$ can control what set of polynomials will be orthogonal in this equation.

To find the Jacobi polynomial set, we will be using $w(x)$ $(1-x)^\alpha(1+x)^\beta$, resulting in formula (1). Values α and β are constant values we can set to change the shape of the polynomial which both need to be greater than -1. When solving the integral, this results in the general equation

$$J_n^{(\alpha,\beta)}(x) = 2^{-n} \sum_{j=0}^n \binom{n+\alpha}{j} \binom{n+\beta}{n-j} (x-1)^{n-j} (x+1)^j \quad (3) \quad [7]$$

3.2 Jacobi Polynomial Set in ANN

For the approach in this paper, we will be using sets of Jacobi polynomial functions using different parameter sets and comparing the average accuracies and computational time against each other. In addition, the base ANN will also be tested to act as a control group for the experiment. We will use the solution to the Jacobi polynomial set as the activation function for each unit. These functions will be used only in the hidden layer, the input layer and output layer will have standard nodes. Each hidden node will use a different polynomial degree starting from 1 up to n number of hidden nodes.

The network used in this paper will use a Feedforward Neural Network (FNN) and use the stochastic gradient decent algorithm to search through the hypothesis space. To combat overfitting, there will be a portion of the test data used as verification that will score the network. Then, the sets of weights of the highest accuracy are then stored and used for final analysis.

In formula (1) the range of possible values for this set of polynomials to be orthogonal is between -1 and 1. To keep the value x between this range, this paper will implement another activation function before the Jacobi functions to bound the ranges and can be differentiated at all points.

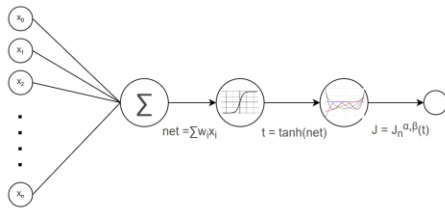


Figure 1: shows a model of a neuron in the ANN on the hidden layer. Notice two activation functions are shown. The first one uses tanh and the second uses Jacobi Polynomial function.

Here we use hyperbolic tangent ($\tanh(x)$) which will bound the values of the net function to $(-1,1)$. The derivative is easy to calculate here with $\frac{d(\tanh(x))}{dx} = \text{sech}^2(x) = 1 - \tanh^2(x)$. Figure 1 shows how the tanh function will be used as the first activation function and the Jacobi Polynomial Function will be the second activation function.

This paper will implement the recurrence formula for the Jacobi Polynomial. This can be generalized using the recurrence formula [7]:

$$J_{n+1}^{\alpha,\beta}(x) = (a_n^{\alpha,\beta} - b_n^{\alpha,\beta})J_n^{\alpha,\beta}(x) - c_{n-1}^{\alpha,\beta}J_{n-1}^{\alpha,\beta}(x), \quad n \geq 1$$

$$J_0^{\alpha,\beta}(x) = 1, \quad J_1^{\alpha,\beta}(x) = \frac{1}{2}(\alpha + \beta + 2)x + \frac{1}{2}(\alpha - \beta) \quad (4)$$

where

$$a_n^{\alpha,\beta} = \frac{(2n+\alpha+\beta+1)(2n+\alpha+\beta+2)}{2(n+1)(n+\alpha+\beta+1)} \quad (4a)$$

$$b_n^{\alpha,\beta} = \frac{(\beta^2 - \alpha^2)(2n+\alpha+\beta+1)}{2(n+1)(n+\alpha+\beta+1)(2n+\alpha+\beta)} \quad (4b)$$

$$c_n^{\alpha,\beta} = \frac{(n+\alpha)(n+\beta)(2n+\alpha+\beta+2)}{(n+1)(n+\alpha+\beta+1)(2n+\alpha+\beta)} \quad (4c)$$

Solving for (4a), (4b) and (4c) can be done at the development stage when building the network, once we define values for α and β . We can store the a , b and c values for each level $(1, \dots, n)$ in the polynomial. Using this formula contains some benefits. First is that it removes all factorials and quotients used in formula (3) which can reduce computation time. In addition, the recursion formula can be solved linearly if we store an array of calculated values and referencing them when needed. The problem is that the computation time still increases as we increase the degree of the polynomial. Larger polynomials tend to have more flexibility on how the formula will be shaped but at the expense of longer computation time as. In addition, the training is longer to learn the function's complexity.

When using backpropagation, we need to use the derivative of the Jacobi Polynomial, which is defined as [7]:

$$\partial_x J_n^{\alpha,\beta}(x) = \frac{1}{2}(n+\alpha+\beta+1)J_{n-1}^{\alpha+1,\beta+1}(x)$$

Therefore, we need to store additional a , b and c values for the derivative. Computational space has significantly increased, but it is worth it for the decrease in time complexity.

3.3 Empirical Evaluation

This paper will evaluate the efficiency and accuracy of the ANN versus the ANN modified with Jacobi Polynomial as the activation function. This paper should show the time loss of increasing complexity with the Jacobi Polynomials but should make it up with the speed in finding the minimum and high percentage accuracy to the test set. The dataset used will be reformatted to run properly with the current code for the ANN. Code for these will be provided in [8].

3.4 Evaluation Criteria

We will observe the different effects of the different networks and compare results. Speed comparisons will be compared in terms of computation time to learn the set and the time it takes to find a minimum value. The timer is set and output for the time it takes to build a network and the time it takes to learn from the training set based on either threshold values or iteration numbers.

3.5 Experimental data and procedures

We will use the database on “Default Credit Card Clients” [4] and “Breast Cancer Wisconsin.” [5] to evaluate the efficiency and accuracy of the networks. The “Default Credit Card Clients” database stores Taiwan user information and predicts if the individual will receive default payments or not. This includes description of the individual (age, sex, marriage status, etc.) and the individual’s previous payments. Attributes in this database are stored as integers. The second, “Breast Cancer Wisconsin,” stores information on the person having cancer based on observing characteristics of 3 different cell nuclei. All attributes in this data are continuous values.

Each database will run with each network over 250 iterations. The current network’s accuracy and the time it took to run will be outputted as data for each 10 iterations. Since the breast cancer database is smaller, we can run a bunch of programs to test computational complexity. This paper will view the set-up time and training time for each network based on time it took to build the network and time it took to train the network. There will be two types of tests: threshold test and hidden node test. Threshold test will run the training program until the program’s accuracy reaches a certain threshold value. Then it stops and outputs the time. The hidden node test will test the time it took to train a network using a set number of iterations but change the number of hidden nodes.

3.6 Results and analysis

After testing out different alpha values and beta values for the network, it did not change the output for the network. Overall, the data shows some benefits to using the Jacobi polynomials over the standard model, but it does not generalize much better than the standard ANN algorithm. Not only that, the Jacobi Polynomial ANN falls into the same problems as the standard ANN.

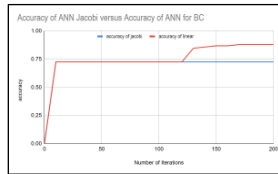


Figure 2: The accuracy results of the Jacobi ANN and the standard ANN on “Breast Cancer Wisconsin” [5] data set. Here, alpha is set to 2 and beta is set to 1 for the Jacobi polynomial. Number of hidden nodes is set to 5

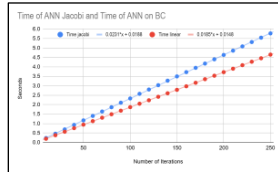


Figure 3: The time results of the Jacobi ANN and the standard ANN on “Breast Cancer Wisconsin” [5] data set. Here, the alpha is set to 3 and the beta is set to 2 for the Jacobi polynomial. Number of hidden nodes is set to 5

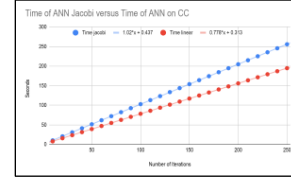


Figure 4: The time results of the Jacobi ANN and the standard ANN on “Default of Credit Card Clients” [4] data set. Here, the alpha is set to 3 and the beta is set to 2 for Jacobi polynomial. Number of hidden nodes is set to 5

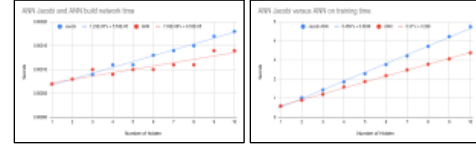


Figure 5: The time results for building the network and training it based on number of hidden nodes in the hidden layer. Here, the alpha is set to 3 and beta is set to 2 in the Jacobi Polynomial

Figures 2 to Figure 4 notes the times or accuracies every 10 iterations. For linear graphs, a best fit line will approximate the growth of the graph with respect to the different networks used.

Looking at figure 2, we can see the limitations of the Jacobi Polynomial. Even with multiple iterations and tests, the ANN using Jacobi Polynomial faced a minimum local problem. The network was unable to jump past a certain threshold, but at some point, the Standard ANN was able to jump past it after several iterations. After testing the accuracy based on the “Default of Credit Card Clients” [4] database, accuracies result was the same between the networks, staying at a consistent 0.774% accuracy.

Figure 3 and figure 4 shows a linear time increase for the Jacobi Polynomial network over the standard network by a factor of 1.25 and 1.31 respectively. Though the factor of computation for the Jacobi Polynomials increased, we still can’t determine the overall efficiency of the algorithm. To find that we must consider the speed at which the networks find the highest accuracy of the model. Figure 2 only shows a jump from zero accuracy to the highest, which means it took less than 10 iterations to reach a local minimum for the error. This is where we are outputting training time after the networks reaches a certain threshold value. For these networks, it got stuck on above 70% which is where the threshold value will be.

ANN using Jacobi Polynomials

training time	0.0252	0.0246	0.026	0.0252	0.0252	0.0255	0.0286	0.0251	0.025	0.0246
# of iterations	1	1	1	1	1	1	1	1	1	1

Standard ANN

training time	0.2098	0.0207	0.0411	0.0209	0.0204	0.0838	0.021	0.409	2.335	0.0207
# of iterations	11	1	2	1	1	4	1	22	128	1

Figure 5: Results shown for the time and number of iterations if took for the networks to reach a certain threshold. This was ran 10 times for each network

From figure 5, the network using Jacobi Polynomials was able to quickly and consistently find a high accuracy value after 1 iteration with an average time of 0.0255 seconds while the standard network model was able to find it after on average 17.2 iterations with an average time of 0.31824 seconds.

4. Conclusion

With the results shown, Jacobi polynomials show promise in its speed. Time complexity of implementing the Jacobi algorithm only increases the time by a factor of around 1.28 instead of exponentially. Which means that this algorithm grows around the same speed as the standard ANN with larger neural networks. Even though computation times increased, it was able to, on average, beat the average time of a standard ANN to find a local minimum, at a cost. A problem with speed is its capability of adaptation. Standard ANN are very chaotic, which we can see in figure 5. This allows the network to prevent itself from being stuck at a local minimum and reduce the risk of the local minimum problem as seen in figure 2. This is why the ANN using the Jacobi was not able to bypass the local minimum problem. In addition, the Jacobi function includes two extra arbitrary values (α and β) that we need to define to control the shape of this function. These values are hard to determine with larger and larger datasets. Multiple test will need to be conducted to see the polynomial shape that fits the best in a given dataset

In future work we can try other methods of reducing the minimum local problem. A combination of hidden layers using both Standard and Jacobi Polynomial activation functions can be used to better shape out the networks as it learns. Another problem is determining α and β values for the network. If possible, they can be learned as a set of weights in a network.

REFERENCES

- [1] H. S. Chakraborty, T. Miezeki, M. Oura, and Y. Tanaka, 'Jacobi polynomials and design theory I', *Discret. Math.*, vol. 346, p. 113339, 2022.
DOI: 10.48550/arXiv.2207.10911
- [2] Chu Kwong Chak, G. Feng and Chi Ming Cheng, "Orthogonal polynomials neural network for function approximation and system modeling," *Proceedings of ICNN'95 - International Conference on Neural Networks*, Perth, WA, Australia, 1995, pp. 594-599 vol.1,
DOI: 10.1109/ICNN.1995.488246.
- [3] H. Zhang, R. Yu, J. Yu, W. Liu, S. Wu, and D. Liu, 'Risk Assessment of Substation based on Hermite orthogonal basis Feedforward Neural Network data Fusion algorithm', in *Proceedings of the 2024 International Conference on Power Electronics and Artificial Intelligence*, Xiamen, China, 2024, pp. 527–531.
DOI: <https://doi.org/10.1145/3674225.3674320>
- [4] "Default of Credit Card Clients," UC Irvine Machine Learning Repository, This research aimed at the case of customers' default payments in Taiwan and compares the predictive accuracy of probability of default among six data mining methods, 4/2/2025 [Online]. Available: <https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients>
- [5] "Breast Cancer Wisconsin (Diagnostic)," UC Irvine Machine Learning Repository, Diagnostic Wisconsin Breast Cancer Database. 4/2/2025 [Online]. Available: <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>
- [6] Szego, Gabor, "Orthogonal Polynomials" in *American Mathematical Society* Providence, RI, United States of America 1939
DOI: <https://people.math.osu.edu/nevai.1/SZEGO/szego=szego1975=ops=OCR.pdf>
- [7] Shen, Jie, et al. "Orthogonal polynomials and related approximation results." *Spectral Methods: Algorithms, Analysis and Applications* (2011): 70-80
DOI: <https://sec.cc.ac.cn/~hyu/teaching/shonm2013/STWchap3.2p.pdf>

- [8] Duong, P 2025 Machine-Learning-CSE5693: Machine-Learning-CSE/Term Paper. [accessed 2025 April 29]. {Link: GitHub - <https://github.com/skyfall1818/Machine-Learning-CSE/tree/main/Term%20Paper>}